

CSE 3204
Formal Language, Automata and Computability



Lecture 22
Turing Machine

Multistack Machine

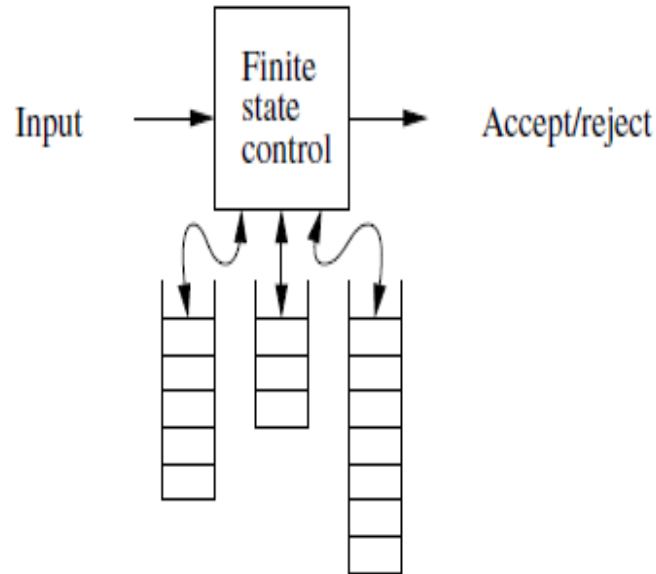


Figure 8.20: A machine with three stacks

- A k-stack machine is a deterministic PDA with k stacks.
- Like the PDA, it obtains its input from an input source.
- Has a finite control, which is one of a finite set of states.
- Has a finite stack alphabet.
- A move in a multistack machine depends on:
 1. The state of the finite control.
 2. The input symbol read, which is chosen from the finite input alphabet.
Alternatively, the multistack machine can make a move using ϵ input, but to make the machine deterministic, there cannot be a choice of an ϵ -move or a non- ϵ -move in any situation.
 3. The top stack symbol on each of its stacks.

Multistack Machine

In one move, the multistack machine can:

- a) Change to a new state.
- b) Replace the top symbol of each stack with a string of zero or more stack symbols. There can be (and usually is) a different replacement string for each stack.

Thus, a typical transition rule for a k -stack machine looks like:

$$\delta(q, a, X_1, X_2, \dots, X_k) = (p, \gamma_1, \gamma_2, \dots, \gamma_k)$$

- Accepts by entering a final state.
- An end marker \$ is kept at the end of the input string. Not part of the input alphabet.

Multistack Machine

Theorem 8.13: If a language L is accepted by a Turing machine, then L is accepted by a two-stack machine.

- A two stacks can simulate one TM tape, with one stack holding the content to the left of the head and the other stack holding content to the right of the head.
- The two stack machine S simulating a TM M will perform the following:
 1. S begins with a *bottom-of-stack marker* on each stack. This marker can be the start symbol for the stacks, and must not appear elsewhere on the stacks. In what follows, we shall say that a “stack is empty” when it contains only the bottom-of-stack marker.
 2. Suppose that $w\$$ is on the input of S . S copies w onto its first stack, ceasing to copy when it reads the endmarker on the input.
 3. S pops each symbol in turn from its first stack and pushes it onto its second stack. Now, the first stack is empty, and the second stack holds w , with the left end of w at the top.
 4. S enters the (simulated) start state of M . It has an empty first stack, representing the fact that M has nothing but blanks to the left of the cell scanned by its tape head. S has a second stack holding w , representing the fact that w appears at and to the right of the cell scanned by M 's head.

Multistack Machine

5. S simulates a move of M as follows.

- (a) S knows the state of M , say q , because S simulates the state of M in its own finite control.
- (b) S knows the symbol X scanned by M 's tape head; it is the top of S 's second stack. As an exception, if the second stack has only the bottom-of-stack marker, then M has just moved to a blank; S interprets the symbol scanned by M as the blank.
- (c) Thus, S knows the next move of M .
- (d) The next state of M is recorded in a component of S 's finite control, in place of the previous state.
- (e) If M replaces X by Y and moves right, then S pushes Y onto its first stack, representing the fact that Y is now to the left of M 's head. X is popped off the second stack of S . However, there are two exceptions:
 - i. If the second stack has only a bottom-of-stack marker (and therefore, X is the blank), then the second stack is not changed; M has moved to yet another blank further to the right.
 - ii. If Y is blank, and the first stack is empty, then that stack remains empty. The reason is that there are still only blanks to the left of M 's head.

Multistack Machine

OF THE STACK.

- (f) If M replaces X by Y and moves left, S pops the top of the first stack, say Z , then replaces X by ZY on the second stack. This change reflects the fact that what used to be one position left of the head is now at the head. As an exception, if Z is the bottom-of-stack marker, then M must push BY onto the second stack and not pop the first stack.
6. S accepts if the new state of M is accepting. Otherwise, S simulates another move of M in the same way.

Counter Machine

A *counter machine* may be thought of in one of two ways:

1. The counter machine has the same structure as the multistack machine (Fig. 8.20), but in place of each stack is a counter. Counters hold any nonnegative integer, but we can only distinguish between zero and nonzero counters. That is, the move of the counter machine depends on its state, input symbol, and which, if any, of the counters are zero. In one move, the counter machine can:
 - (a) Change state.
 - (b) Add or subtract 1 from any of its counters, independently. However, a counter is not allowed to become negative, so it cannot subtract 1 from a counter that is currently 0.
2. A counter machine may also be regarded as a restricted multistack machine. The restrictions are as follows:
 - (a) There are only two stack symbols, which we shall refer to as Z_0 (the *bottom-of-stack marker*), and X .
 - (b) Z_0 is initially on each stack.
 - (c) We may replace Z_0 only by a string of the form $X^i Z_0$, for some $i \geq 0$.
 - (d) We may replace X only by X^i for some $i \geq 0$. That is, Z_0 appears only on the bottom of each stack, and all other stack symbols, if any, are X .

Counter Machine

We shall use definition (1) for counter machines, but the two definitions clearly define machines of equivalent power. The reason is that stack $X^i Z_0$ can be identified with the count i . In definition (2), we can tell count 0 from other counts, because for count 0 we see Z_0 on top of the stack, and otherwise we see X . However, we cannot distinguish two positive counts, since both have X on top of the stack.

Theorem 8.14: Every recursively enumerable language is accepted by a three-counter machine.

Proof from the book.

Counter Machine

Theorem 8.15 : Every recursively enumerable language is accepted by a two-counter machine.

- Proof from the book.

- Section 8.5 of Chapter 8